



Grizon AI — Autonomous Security Operating System

Final Unified Architecture & Execution Blueprint

Version: 2.0

Status: Strategic Architecture Document

Goal: Build an AI-powered autonomous security platform capable of understanding, testing, validating, patching, and certifying software repositories.



Executive Vision

Grizon AI is not just a code scanner.

It is designed to become:

“An Autonomous Security Operating System for Software.”

The platform combines:

- Static Analysis
- Semantic Code Understanding
- Dynamic Runtime Testing
- AI Multi-Agent Reasoning
- Adaptive Security Test Generation
- Autonomous Patch Generation
- Performance Validation
- Production Certification

into one unified architecture.



Core Philosophy

Most existing security tools:

- depend heavily on one AI provider
- rely on regex-based scanning
- generate excessive false positives
- cannot understand business logic
- fail to detect workflow abuse
- cannot validate runtime behavior

Grizon solves this using:

1. Deterministic Security Engines

Fast local scanning engines for zero-cost baseline validation.

2. Multi-Provider AI Orchestration

Uses multiple AI providers for resilience, cost optimization, and specialized reasoning.

3. Dynamic Behavioral Validation

Tests real application behavior instead of only source code patterns.

4. Adaptive Security Intelligence

Generates context-aware attack simulations dynamically from application architecture.

High-Level System Architecture

```
graph TD; A[User Repository Upload] --> B[Repository Ingestion Engine]; B --> C[Secure Sandbox Environment]; C --> D[Static Deterministic Analysis]; D --> E[Repository Intelligence Graph]; E --> F[Dynamic Workflow Mapping]; F --> G[AI Multi-Agent Security Review]; G --> H[Adaptive Attack Simulation]; H --> I[Autonomous Remediation Engine]; I --> J[Revalidation Pipeline]; J --> K[GSS Security Certification];
```

Phase 1 — Secure Sandbox & Execution Infrastructure

Goal

Safely execute untrusted repositories without compromising infrastructure.

Recommended Infrastructure Stack

Layer	Technology
Isolation Layer	Firecracker MicroVM
Runtime Layer	Docker
Browser Terminal	xterm.js
PTY Backend	node-pty
Browser Automation	Playwright
Load Testing	k6
Queue System	Redis / RabbitMQ
Storage	PostgreSQL + Object Storage
Runtime Monitoring	eBPF / OpenTelemetry

Sandbox Security Controls

Each repository executes inside:

Firecracker MicroVM

Inside VM:

- Docker runtime
 - browser runtime
 - scanners
 - AI execution agents
 - isolated filesystem
-

Required Security Controls

Security Control	Purpose
CPU Limits	Prevent abuse
Memory Limits	Prevent crashes
Disk Quotas	Resource isolation
Network Isolation	Block malicious traffic
Read-only Base Images	Prevent persistence
Seccomp Profiles	Syscall restriction
AppArmor/SELinux	Process confinement
Auto VM Cleanup	Ephemeral environments
Timeout Enforcement	Infinite loop protection

Phase 2 — Deterministic Security Engines

Goal

Perform fast, scalable, low-cost baseline security analysis.

These engines provide:

- immediate findings
- repository intelligence
- AI context compression
- low operational cost

Layer 1 — Secret & Entropy Scanner

Detects

- AWS keys
- API tokens
- JWT secrets
- DB credentials
- Stripe keys
- SSH keys
- hardcoded passwords

Detection Methods

Pass 1 — Regex Matching

High-confidence signature detection.

Pass 2 — Entropy Analysis

Shannon entropy detection for unknown secrets.

Output Classification

Severity	Meaning
Critical	Direct secret exposure
Warning	Suspicious high-entropy values

Layer 2 — AST & Semantic Static Analysis

Technology

- tree-sitter
 - AST parsing
 - S-expression queries
-

Detects

Vulnerability	Example
SQL Injection	User input → DB query
OS Injection	Input → system command
Weak Crypto	md5 / sha1
Missing Auth	Missing middleware
Unsafe Deserialization	Arbitrary object execution
Dangerous Functions	eval / exec

Why AST Instead of Regex

AST understands:

- scopes
- functions
- imports
- variable flow
- execution structure

This reduces false positives significantly.

Layer 3 — Advanced Data Flow & Taint Analysis

Goal

Understand how user-controlled data moves through the application.

Core Components

Component	Purpose
CFG	Control Flow Graph
DFG	Data Flow Graph
Call Graph	Function relationships
Symbol Table	Variable ownership
Interprocedural Analysis	Cross-file tracking

Detection Logic

```
graph TD; A[Untrusted Input] --> B[Variable Propagation]; B --> C[Dangerous Sink]; C --> D[Security Finding];
```

Example

```
user_input = request.args.get("q")
query = "SELECT * FROM users WHERE name='" + user_input + "'"
```

Grizon identifies:

- untrusted source
- variable propagation
- vulnerable sink

→ SQL Injection.

Layer 4 — Local Vulnerability Intelligence DB

Purpose

Detect vulnerable dependencies without relying on external paid services.

Data Sources

Source	Purpose
NVD	CVE database
OSV.dev	Open-source vulnerabilities
GHSA	GitHub advisories

Features

- local PostgreSQL mirror
 - SemVer matching
 - dependency graph analysis
 - reachability analysis
-

Reachability Analysis

Instead of:

"Library vulnerable."

Grizon asks:

"Is vulnerable code actually imported or reachable?"

This dramatically reduces false positives.

Phase 3 — Repository Intelligence Layer

Goal

Understand repository architecture before involving expensive AI models.

Repository Graph Engine

Grizon builds:

Graph	Purpose
Route Graph	API relationships
Auth Graph	Authentication flow
Service Graph	Microservice interactions
Dependency Graph	Package structure
Workflow Graph	Business logic states
Runtime Graph	Execution behavior

Critical Path Detection

The system automatically identifies:

- authentication logic
- payment systems
- admin APIs
- OTP flows
- RBAC logic
- session management
- sensitive workflows

Only critical paths are escalated to advanced AI review.

Phase 4 — Dynamic Behavioral Validation

Goal

Detect real-world exploitable vulnerabilities through runtime testing.

Problem With Traditional Scanners

Most vulnerabilities are:

- workflow flaws
- logic bypasses
- race conditions
- state manipulation
- privilege escalation

These cannot be reliably detected using regex alone.

Dynamic Runtime Mapping

During execution Grizon observes:

- routes
 - middleware
 - sessions
 - DB queries
 - auth validation
 - request flows
 - state transitions
-

Runtime Knowledge Graph

```
graph TD;
  A[User Request] --> B[Middleware];
  B --> C[Controller];
  C --> D[Service Layer];
  D --> E[Database];
```

Adaptive Security Test Generation

Instead of static hardcoded tests:

Grizon dynamically generates attack scenarios.

Example Workflow Discovery

Detected Flow:

```
/login  
/verify-otp  
/reset-password
```

Generated Tests:

Test	Purpose
Skip OTP	Auth bypass
Replay OTP	Replay attack
Step order manipulation	Workflow abuse
Expired token reuse	Session flaw
Concurrent reset	Race condition

Behavioral Security Validation

The system validates:

Validation Type	Example
Authentication	Unauthorized access
Authorization	Cross-user access
API Security	Injection payloads
Payments	Negative amount attacks
Rate Limiting	Abuse protection
Session Security	Replay attacks
Upload Security	Path traversal

Validation Type	Example
Business Logic	Workflow manipulation

Phase 5 — AI Multi-Agent Security System

Goal

Use specialized AI reasoning agents instead of one generic model.

Multi-Agent Architecture

Agent	Responsibility
Auth Agent	Authentication flaws
RBAC Agent	Authorization review
API Agent	Endpoint security
Crypto Agent	Encryption review
Logic Agent	Business workflows
Dependency Agent	CVE reasoning
Performance Agent	Optimization analysis
Patch Agent	Secure remediation

Why Multi-Agent?

Benefits:

- reduced hallucinations
 - focused reasoning
 - specialized prompts
 - scalable analysis
 - better accuracy
-

Multi-Provider AI Orchestration

Supported Providers

Provider	Purpose
OpenAI	General reasoning
Claude	Deep security analysis
Gemini	Fast triage
DeepSeek	Cost-efficient filtering
xAI	Secondary audit review

AI Routing Strategy

Tier	Purpose	Models
Tier 1	Cheap filtering	DeepSeek / Gemini Flash
Tier 2	Medium reasoning	GPT-4o / Grok
Tier 3	Critical analysis	Claude Sonnet

Consensus Engine

Multiple AI systems independently review findings.

```
Static Finding
  ↓
Agent 1 Review
  ↓
Agent 2 Review
  ↓
Consensus Decision
  ↓
Escalation or Dismissal
```

This reduces false positives dramatically.

Phase 6 — Autonomous Remediation Engine

Goal

Automatically generate secure patches.

Remediation Pipeline

```
graph TD; A[Security Finding] --> B[Context Extraction]; B --> C[Patch Generation]; C --> D[Sandbox Revalidation]; D --> E[Loop Until Clean];
```

Patch Validation

After AI patch generation:

- deterministic scanners rerun
- behavioral tests rerun
- performance tests rerun
- security validations rerun

If issue persists:

→ remediation loop repeats.

Phase 7 — Performance & Runtime Validation

Goal

Ensure application is production-ready.

Frontend Analysis

Uses:

- Lighthouse
- Puppeteer
- Playwright

Measures:

- performance
 - accessibility
 - SEO
 - render blocking
 - hydration issues
-

Backend Analysis

Uses:

- k6
- runtime profiling
- flame graph analysis

Measures:

- p99 latency
 - throughput
 - concurrency bottlenecks
 - memory usage
 - DB query performance
-



Phase 8 — GSS (Grizon Security Score)

Goal

Determine if repository is production ready.

GSS Formula

Category	Weight
Security	40%

Category	Weight
Performance	20%
Code Quality	20%
Compliance	20%

Certification Rules

Score	Status
<95	Blocked
≥95	Production Certified

Certification Deliverables

Generated Artifacts:

- security-audit.pdf
- vulnerability report
- patch summary
- runtime analysis
- performance metrics
- certification manifest

Long-Term Vision

Grizon evolves from:

“Static Code Scanner”

into:

“Autonomous AI Security Research System.”

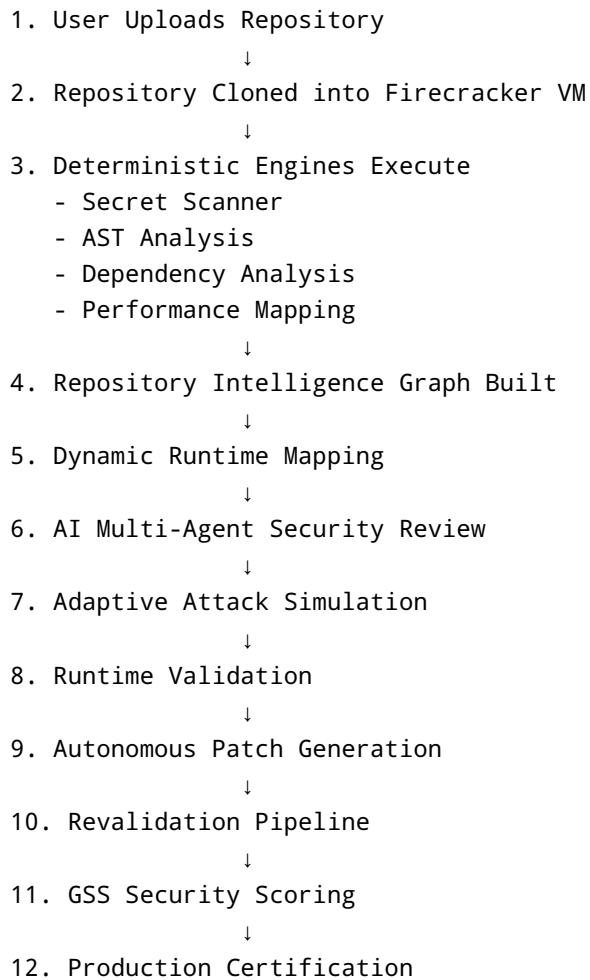
The platform ultimately becomes capable of:

- understanding repositories
- modeling workflows
- simulating attackers
- dynamically generating exploits

- validating runtime behavior
 - autonomously healing vulnerabilities
 - certifying production readiness
-



Final Unified Execution Flow



Final Statement

Grizon AI is designed to become:

“The AI-Native Security Operating System for Modern Software.”

It combines:

- deterministic security analysis
- graph intelligence
- runtime behavioral validation
- adaptive attack generation
- AI multi-agent reasoning
- autonomous remediation
- production certification

into one unified autonomous platform.

This architecture positions Grizon beyond traditional DevSecOps tools and toward the future of autonomous software security engineering.